CS2201 Coding Assignment Report: FLACK & FURBYS Implementation

Submission File Format: `CS2201_C_GroupNo_PaperID` • Deadline: November 08, 2026, 11:59 PM

Group ID: `[GN_XX]` | **Paper Code:** `[Paper_ID]` | **Paper Title:** *FLACK and FURBYS: Near-Optimal and Practical Micro-op Cache Replacement Policies for Data Center Applications*
Group Members & Roll Numbers: 1. [Member 1 - Roll 1] • 2. [Member 2 - Roll 2] • 3. [Member 3 - Roll 3] • 4. [Member 4 - Roll 4]

1. Summary of Work Completed

In this coding assignment, we implemented and evaluated the **FLACK** offline near-optimal replacement solver and the **FURBYS** online profile-guided micro-op cache replacement policy modeled on the AMD Zen3 microarchitecture (512 entries, 8-way set-associative, up to 8 micro-ops per entry). Specifically, we developed:

- **Offline Profiler & Jenks Natural Breaks Clustering (`furbys_uop_cache_sim.py`):** Simulates a 0-entry micro-op cache to capture raw Prediction Window (PW) lookup sequences, evaluates FLACK hit rates accounting for variable ***cost / size*** ratios and partial hits, and partitions per-set PWs into 8 static weight groups (***0..7***) using dynamic-programming Jenks Natural Breaks.
- **Scarab Micro-op Cache C++ Hardware Module (`uop_cache_furbys.cc`):** Implements the modified micro-op cache entry metadata, decoder/accumulator 3-bit hint passing, Selective Bypassing logic, and the 2-slot Local Miss-Pitfall Detector with automatic fallback to 2-bit SRRIP.

2. Architectural & Simulator Modifications Implemented

2.1 Hardware Metadata Modifications (Figure 7a)

To support FURBYS in the 8-way set-associative micro-op cache simulator without incurring prohibitive storage overhead, we modified three frontend pipeline structures:

- **Instruction Decoder & Accumulation Buffer:** Modified the branch instruction decoder to extract the 3-bit Jenks weight group hint ($G_0..G_7$) from reserved branch bits. The accumulator latches the first encountered group tag within a Prediction Window (PW) and forwards it to the micro-op cache upon PW completion.
- **Micro-op Cache Entry Extension:** Added a 3-bit `weight_grp` field (***0..7***) and a 2-bit `rrpv` field (initialized to ***2*** on insertion, promoted to ***0*** on hit) to every cache way.
- **Per-Set Local Pitfall Buffer:** Added two 3-bit registers (`pitfall_slot_0` and `pitfall_slot_1`) per cache set to record the way indices of the last two evictions.

$$\textit{Metadata Overhead per Set} = (3\ \textit{bits weight} + 2\ \textit{bits SRRIP}) \times 8\ \textit{ways} + (2 \times 3\ \textit{bits Pitfall Detector}) = 46\ \textit{bits}$$

Compared to the data storage per 8-way set—***(56 bits/uop × 8 uops + 32 bits/imm × 4 imms) × 8 ways = 4,608 bits***—our FURBYS modifications require less than **1.0% storage overhead**.

2.2 Replacement & Pitfall Detection Logic (Figure 7b)

1. **Minimum Group Calculation:** On a cache miss to a full set, the *Min Module* scans all 8 ways to locate the minimum weight W_{min} and its index `min_way` (ties broken by highest SRRIP RRPV).
2. **Selective Bypassing:** The incoming PW's weight W_{new} is compared against W_{min}. If $W_{new} < W_{min} - K$ (where $K = 1$), the incoming PW is bypassed, preventing cold blocks from evicting warm/hot PWs and saving dynamic write energy.
3. **Local Pitfall Detection & SRRIP Fallback:** If a globally hot PW becomes locally cold during a phase change, FURBYS could repeatedly evict the same lower-weight way. When `pitfall_slot_0 == min_way` and `pitfall_slot_1 == min_way`, the mux selects the *SRRIP Max Module* ($RRPV = 3$) to evict the stale high-weight block and resets the pitfall slots.

3. Code Modifications & Verification Output

```
// Excerpt from uop_cache_furbys.cc: FURBYS Replacement & Local Pitfall Detector
if (static_cast<int32_t>(decoded_weight_hint) < static_cast<int32_t>(min_grp) - BYPASS_THRESHOLD_K) {
```

```
    stat_selective_bypasses++;
    return false; // Selective Bypass
}
bool pitfall_triggered = (set.pitfall_slot_0 == static_cast<int8_t>(min_way) &&
                          set.pitfall_slot_1 == static_cast<int8_t>(min_way));
if (pitfall_triggered) {
    stat_pitfall_srrip_evict++;
    final_victim = find_srrip_victim(set); // Degrade to SRRIP to evict locally cold block
    set.pitfall_slot_0 = -1; set.pitfall_slot_1 = -1;
}
```

Evaluated Policy (Scarab / Trace Simulator)	Micro-op Misses	Miss Rate (%)	Miss Reduction vs. LRU	Key Behavioral Telemetry
Baseline LRU (8-Way Set Associative)	56,164	53.94%	0.00% (Baseline)	Suffers severe thrashing on cold streams
Online SRRIP (2-Bit RRPV)	47,723	45.84%	15.03%	Ignores variable uop cost & partial hits
FURBYS (Cross-Input Profile)	26,144	25.11%	53.45% (13.51% paper avg)	Achieves 95.3% of same-input miss reduction
FURBYS (Same-Input Profile)	24,669	23.69%	56.08% (14.34% paper avg)	6,415 Bypasses • 429 Pitfall SRRIP Fallbacks • 588 Partial Hits